Unit 1.4 - Floating-Point Arithmetic

1 Motivation for Finite Precision Arithmetic

Computers are inherently limited in the amount of memory they possess, which means they cannot represent numbers with infinite precision. For example, the mathematical constant π is an irrational number with an infinite number of decimal places:

$$\pi = 3.14159265358979323846\ldots$$

However, since a computer cannot store an infinite sequence of digits, π must be approximated using a finite number of digits. This necessity of finite representation is true for all real numbers in computing. As a result, computers use **floating-point arithmetic**, a way of representing real numbers that can handle a wide range of values but only with limited precision.

Floating-point numbers are stored in a computer's memory using a finite number of bits (binary digits). Since each bit can only represent a 0 or a 1, only a finite set of numbers can be represented exactly. Any number that cannot be represented exactly must be approximated, leading to **rounding errors** in calculations.

2 Why Base 2 Representation?

The choice of base 2 (binary) representation in computing is motivated by the physical structure of digital systems. Digital computers are built from electronic components that can reliably exist in one of two states (off or on), which are naturally represented by the binary digits 0 and 1. This binary system is both simple and efficient for hardware implementation, as it corresponds directly to the on/off state of transistors in the computer's circuitry.

On the other hand, base 10 (decimal) is more intuitive for humans because we have ten fingers, which historically led to the development of the decimal numbering system. Implementing base 10 in computer hardware would be more complex and less efficient than using base 2 because it would require distinguishing between ten different states, as opposed to just two.

3 Converting Between Decimal and Binary

3.1 Converting Decimal to Binary

To convert a decimal (base 10) number to binary (base 2), we use two main methods: the method for converting integers and the method for converting fractional parts.

3.1.1 Converting a Decimal Integer to Binary

To convert a decimal integer to binary, repeatedly divide the number by 2 and record the remainder for each division. The binary representation is obtained by reading the remainders from bottom to top.

Example: Convert 25 to Binary Let's convert the decimal number 25 to binary:

Quotient	Remainder
$25 \div 2 = 12$	1
$12 \div 2 = 6$	0
$6 \div 2 = 3$	0
$3 \div 2 = 1$	1
$1 \div 2 = 0$	1

By reading the remainders from bottom to top, the binary representation of 25 is:

$$25_{10} = 11001_2.$$

3.1.2 Converting a Decimal Fraction to Binary

To convert the fractional part of a decimal number to binary, multiply the fractional part by 2 and record the integer part of the result. Then, take the fractional part of the result and repeat the process. The binary representation is obtained by reading the integer parts from top to bottom.

Example: Convert 0.625 to Binary Let's convert the decimal fraction 0.625 to binary:

Operation	Integer Part
$0.625 \times 2 = 1.25$	1
$0.25 \times 2 = 0.5$	0
$0.5 \times 2 = 1.0$	1

By reading the integer parts from top to bottom, the binary representation of 0.625 is:

$$0.625_{10} = 0.101_2.$$

3.1.3 Combining Integer and Fractional Parts

To convert a decimal number with both integer and fractional parts, convert each part separately and then combine them.

Example: Convert 25.625 to Binary First, convert the integer part 25 to binary as shown earlier: $25_{10} = 11001_2$. Next, convert the fractional part 0.625 to binary as shown earlier: $0.625_{10} = 0.101_2$. Combining both parts, we have: $25.625_{10} = 1101.101_2$.

3.2 Converting Binary to Decimal

To convert a binary number to decimal, multiply each digit by 2 raised to its position number, counting from right to left and starting at zero. Then sum all these values to obtain the decimal representation.

3.2.1 Converting a Binary Integer to Decimal

Example: Convert 11001_2 to Decimal Let's convert the binary number 11001_2 to decimal:

$$\begin{aligned} 11001_2 &= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 16 + 8 + 0 + 0 + 1 \\ &= 25_{10}. \end{aligned}$$

3.2.2 Converting a Binary Fraction to Decimal

Example: Convert 0.101_2 to Decimal Let's convert the binary fraction 0.101_2 to decimal:

$$\begin{aligned} 0.101_2 &= 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= \frac{1}{2} + 0 + \frac{1}{8} \\ &= 0.5 + 0 + 0.125 \\ &= 0.625_{10}. \end{aligned}$$

3.2.3 Combining Integer and Fractional Parts

To convert a binary number with both integer and fractional parts, convert each part separately and then combine them.

Example: Convert 11001.101_2 to Decimal First, convert the integer part 11001_2 to decimal as shown earlier: $11001_2 = 25_{10}$. Next, convert the fractional part 0.101_2 to decimal as shown earlier: $0.101_2 = 0.625_{10}$. Combining both parts, we have: $11001.101_2 = 25.625_{10}$.

4 Floating-Point Intro

In floating-point arithmetic, a real number is typically represented in a format similar to scientific notation. The general form of a floating-point number is:

$$\text{sign} \times \text{mantissa} \times \text{base}^{\text{exponent}},$$

where:

- **Sign** indicates whether the number is positive or negative.
- **Mantissa** represents the significant digits of the number.
- **Base** is the base of the number system (base 2 for binary, 10 for decimal).
- **Exponent** determines the scale or magnitude of the number.

For example, the decimal number 25.625 can be represented in binary floating-point format as:

$$25.625 = +2.5625 \times 10^1$$

where:

- The **sign** is positive
- The **mantissa** is 2.5625
- The **base** is 10
- The **exponent** is 1

The binary number 11001.101 can be represented in binary floating-point format as:

$$11001.101 = +1.1001101 \times 2^4$$

where:

- The **sign** is positive
- The **mantissa** is 1.1001101
- The **base** is 2
- The **exponent** is 4

5 Floating-Point In Detail (from Heath)

A floating-point number system is characterized by four integers:

- β: Base or radix
- p: Precision
- $[L, U]$: Exponent range

A real number x is represented as:

$$x = \pm \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}} \right) \beta^E,$$

where:

- $0 \leq d_i \leq \beta - 1$ for $i = 0, \ldots, p-1$,
- $L \leq E \leq U$ is the range of the exponent E.

The d_i's correspond to the digits of the mantissa. Also, most floating-point systems are *normalized*, meaning that the first digit d_0 cannot be 0.

The following are some special numbers of a floating-point system:

- **Total Number of Normalized Floating-Point Numbers:**

$$2(\beta - 1)\beta^{p-1}(U - L + 1) + 1$$

- **Smallest Positive Normalized Number (UFL, underflow limit):**

$$\text{UFL} = \beta^L$$

- **Largest Floating-Point Number (OFL, overflow limit):**

$$\text{OFL} = \beta^{U+1}\left(1 - \beta^{-p}\right)$$

- **Machine Epsilon (ϵ):**

  The smallest number ϵ such that $\text{fl}(1 + \epsilon) > 1$, where $\text{fl}(x)$ represents the nearest floating point approximation of x

5.1 Limitations of Floating-Point Arithmetic

Because only a finite number of bits are available, not all real numbers can be represented exactly. This leads to several limitations:

- **Precision Loss:** Only a certain number of significant digits can be represented, and numbers that require more digits must be approximated.

- **Rounding Errors:** Operations like addition, subtraction, multiplication, and division may result in small errors due to rounding.
- **Overflow and Underflow:** Numbers that are too large or too small to be represented within the available range result in overflow or underflow.

A particularly interesting case is given by the harmonic series:

$$\sum_{n=1}^{\infty} \frac{1}{n},$$

which diverges (equals ∞) since the sum grows without bound. However, in floating-point arithmetic, the sum may not reflect this divergence accurately due to a number of reasons:

- **Machine precision:** The while the partial sums increase, the indiviaul terms decrease. Thus, machine precision will eventually be reached and there won't be enough digits in the mantissa to compute the sum (consider how adding $10^{10} + 10^{-10}$).
- **Overflow:** Even if machine precision weren'y an issue, there is a maximum limit to the size of floating-point numbers. Since the sum increases without bound, this limit will eventually be reached, causing overflow.

Despite these limitations, floating-point arithmetic is a practical and efficient way to handle real numbers in computer systems, allowing a wide range of values to be represented with reasonable precision.